

Informe Técnico: Arquitectura de Microservicios E-Commerce

1. Introducción

El presente documento describe la arquitectura técnica y las decisiones de diseño tomadas para el desarrollo del sistema de E-Commerce, cuyo objetivo es la implementación de microservicios independientes siguiendo principios de escalabilidad y desacoplamiento.

2. Decisiones de Arquitectura

- **Microservicios Desacoplados:** Cada funcionalidad (Catálogo, Carrito, Órdenes, Usuarios, Notificaciones) se diseñó como una API independiente con su propio dominio de datos.
- **Comunicación:** Se implementó comunicación sincrónica mediante HttpClient (Typed Clients), permitiendo que los servicios de orden y carrito validen consistencia (stock y existencia de usuario) en tiempo real.
- **Persistencia:** Se optó por **SQLite** como motor de base de datos local para cada microservicio, utilizando **Dapper** como micro-ORM. Esta elección garantiza un rendimiento óptimo, transaccionalidad mediante SQL nativo y facilidad de despliegue para entornos de desarrollo.

3. Aspectos Transversales (Cross-Cutting Concerns)

- **Manejo Global de Errores:** Se implementó una capa centralizada mediante `ExceptionHandler` que intercepta excepciones de dominio (`NotFoundException`, `BusinessRuleException`). Todas las respuestas de error siguen el estándar IETF **Problem Details**, garantizando que el cliente reciba un `errorCode` y un `errorMessage` consistentes conforme al catálogo definido por la cátedra.
- **Observabilidad:**
 - **Logging:** Implementación de **Serilog** con arquitectura de doble sink: consola para errores críticos y archivos JSON rotativos para auditoría de requests.
 - **Health Checks:** Se configuraron endpoints `/health/live` (para Kubernetes/Orquestadores) y `/health/ready` (que valida la conexión activa a SQLite), asegurando que el servicio solo reciba tráfico si su base de datos está operativa.
- **Documentación:** Las APIs fueron documentadas mediante **Swagger/OpenAPI**, incluyendo esquemas de datos, respuestas de éxito/error y documentación XML detallada en los endpoints.

4. Conclusión

La arquitectura propuesta permite una separación clara de responsabilidades, facilitando el mantenimiento y la evolución independiente de cada módulo. El sistema cumple con los estándares de robustez, trazabilidad y observabilidad requeridos para una aplicación de grado profesional en .NET Core 8.